

Bluetooth Low Energy: Programação

Tecnologias de Comunicação para a Internet das Coisas:
Sebenta da Aula 05

Catarina Silva

2026

Abstract

Este documento acompanha a quinta aula da unidade curricular e trata o desenvolvimento de aplicações Bluetooth Low Energy: as bibliotecas disponíveis, o modelo de programação da biblioteca bluepy, e a implementação de um servidor BLE em Arduino. Destina-se ao estudo autónomo e ao apoio ao guião prático.

Table of Contents

1 Enquadramento e Bibliotecas

Esta aula passa da descrição do protocolo à sua utilização efetiva. Antes de escrever código, importa perceber que papéis cada peça do exercício desempenha e que ferramentas estão disponíveis.

1.1 Arquitetura do Exercício

O objetivo desta aula é implementar, na prática, os conceitos de GAP e GATT descritos na aula anterior. Do lado do sensor, um Arduino equipado com um shield BLE assume o papel de periférico e, portanto, de servidor GATT: é ele que detém os dados e os expõe. Do lado oposto, um computador com Linux

(que nas aulas seguintes será um Raspberry Pi) assume o papel de central e de cliente GATT: descobre o periférico, estabelece ligação, e lê ou escreve características para obter as leituras dos sensores.

Esta divisão de papéis não é arbitrária. Reproduz a arquitetura típica de uma solução de IoT, em que um nó restrito recolhe dados e um nó mais capaz os agrega e encaminha para a rede. Reter esta correspondência ajuda a situar o exercício no todo da unidade curricular: o que aqui se constrói é o primeiro troço do percurso que vai do sensor ao broker de mensagens.

1.2 Bibliotecas e Wrappers Disponíveis

Em Linux, a base de tudo é o BlueZ, a pilha protocolar Bluetooth oficial do sistema. Suporta todos os protocolos e camadas centrais do Bluetooth, e os pacotes bluez-utils e bluez-firmware fornecem os utilitários de baixo nível e o firmware necessário ao funcionamento dos adaptadores.

Sobre o BlueZ assentam várias bibliotecas de mais alto nível. A bluepy é uma interface Python para BLE e é a biblioteca adotada nesta unidade curricular. A PyGATT constitui uma alternativa em Python, com interface centrada nas operações GATT. A SimpleBLE oferece uma API simplificada, orientada a casos de utilização comuns.

A escolha entre elas condiciona sobretudo o nível de abstração disponível e a facilidade com que se acede a funcionalidades menos frequentes. Importa, porém, reter que todas assentam na mesma pilha subjacente: um problema ao nível do BlueZ ou do adaptador manifesta-se em qualquer delas, e é por aí que deve começar o diagnóstico.

2 Programação com bluepy

Este capítulo apresenta o modelo de programação da biblioteca bluepy e os dois padrões fundamentais de uma aplicação cliente: o varrimento e o tratamento de notificações.

2.1 Modelo de Classes

O modelo de classes da bluepy tem a vantagem de mapear de forma quase direta os conceitos do protocolo, o que reduz a distância entre a teoria e o código.

A classe `Peripheral` representa um dispositivo periférico ao qual o programa se liga, e é o ponto de entrada para todas as operações sobre esse dispositivo. A classe `Scanner` procura dispositivos BLE que estejam a anunciar-se na vizinhança, e cada dispositivo encontrado é representado por um objeto `ScanEntry`, com o respetivo endereço e dados de anúncio. A classe `DefaultDelegate` é a classe base cujos métodos são invocados quando ocorrem eventos assíncronos, como a descoberta de um dispositivo ou a chegada de uma notificação. A classe `UUID` representa os identificadores de serviços e características. Finalmente, as classes `Service`, `Characteristic` e `Descriptor` correspondem diretamente à hierarquia GATT estudada na aula anterior.

2.2 Varrimento de Dispositivos

O varrimento é o primeiro passo de qualquer aplicação cliente: sem descobrir o periférico, não há ligação possível.

O padrão de implementação assenta na delegação. Cria-se uma subclasse de `DefaultDelegate` e redefine-se o método `handleDiscovery`, que recebe o dispositivo encontrado e dois indicadores: se se trata de um dispositivo novo e se traz dados novos. Instancia-se depois um objeto `Scanner` e associa-se-lhe o

delegado através do método `withDelegate`. A invocação de `scan`, com a duração pretendida em segundos, devolve a lista de dispositivos encontrados.

Para cada dispositivo ficam disponíveis o endereço, acessível pelo atributo `addr`, o tipo de endereço e a potência do sinal recebido, o RSSI, expressa em dBm. O RSSI é útil para estimar proximidade relativa entre dispositivos, mas não deve interpretar-se como medida fiável de distância: depende da potência de emissão, da orientação das antenas e dos obstáculos presentes.

2.3 Notificações

Ler periodicamente uma característica obriga o cliente a interrogar o servidor de forma repetida, mesmo quando não há dados novos. A abordagem é simples de programar, mas ineficiente: gasta energia em ambos os dispositivos e introduz latência, já que uma alteração só é detetada na leitura seguinte.

As notificações invertem a iniciativa. É o servidor que informa o cliente sempre que o valor de uma característica se altera. No protocolo distinguem-se dois mecanismos próximos: a notificação, que não exige confirmação do cliente, e a indicação, que exige, sendo por isso mais fiável e simultaneamente mais dispendiosa em tráfego e energia. Este mecanismo é particularmente adequado a sensores que reportam eventos esporádicos, como a deteção de movimento ou a ultrapassagem de um limiar, e é também o mecanismo que sustenta, mais adiante na unidade curricular, a integração com um broker de mensagens.

A implementação em `bluepy` segue o mesmo padrão de delegação usado no varrimento. Define-se uma subclasse de `DefaultDelegate` e redefine-se o método `handleNotification`, que recebe dois argumentos: o *handle* da característica que gerou a notificação e os dados recebidos. O *handle* é o que permite distinguir a origem quando várias características estão configuradas para notificar. Cria-se então o objeto `Peripheral` com o endereço do dispositivo e associa-se o delegado através de `setDelegate`. Obtém-se o serviço pretendido com `getServiceByUUID` e, a partir dele, a característica com `getCharacteristics`. A

notificação é ativada escrevendo no descritor de configuração da característica. O programa entra depois num ciclo de espera, no qual chama `waitForNotifications`; sempre que chega uma notificação, o método `handleNotification` é invocado automaticamente.

3 O Servidor BLE em Arduino

Este capítulo descreve o lado do periférico: a preparação do hardware, a estrutura do programa e a verificação da ligação a partir do gateway.

3.1 Preparação do Hardware

O exercício prático recorre a um shield de comunicações BLE instalado sobre o Arduino, baseado no módulo HM-11. A configuração do modo de funcionamento faz-se através da colocação de *jumpers* no shield, operação que deve ser feita antes da montagem sobre a placa.

Do ponto de vista da ligação ao microcontrolador, o módulo comunica por porta série. Como a porta série física do Arduino está ocupada pela ligação ao computador, usada para carregar o programa e para o monitor série, é necessário definir uma porta série por software, recorrendo à biblioteca `SoftwareSerial`. O pino de *reset* do módulo é controlado por uma saída digital do Arduino, o que permite reinicializá-lo no arranque do programa e garantir um estado conhecido. Ambas as portas série, a física e a virtual, operam a 9600 bit/s.

3.2 Estrutura do Programa

Na função `setup`, o pino de *reset* é configurado como saída e o módulo é reinicializado através de um impulso, colocação do pino em nível baixo, breve atraso, e regresso a nível alto. Seguem-se as inicializações das duas portas série.

Na função loop, o programa verifica se há dados disponíveis vindos do módulo BLE ou do monitor série, e lê o carácter recebido. Este carácter funciona como comando: t para temperatura, h para humidade, l para luminosidade e X para o conjunto de todas as grandezas. A cada comando corresponde uma resposta, enviada simultaneamente para o módulo BLE e para o monitor série, duplicação deliberada, que facilita muito o diagnóstico durante o desenvolvimento, porque permite confirmar no computador o que está a ser enviado por rádio.

Cada comando define ainda um fator de escala aplicado ao atraso do ciclo, o que permite ajustar a periodicidade de resposta consoante a grandeza pedida. Grandezas que variam lentamente, como a temperatura, não justificam a mesma frequência de leitura que grandezas mais voláteis.

3.3 Verificação a partir do Gateway e Método de Trabalho

Do lado do gateway, a verificação faz-se por linha de comandos antes de qualquer desenvolvimento em Python. O comando `hcitool dev` lista os adaptadores Bluetooth disponíveis no sistema, com os respetivos identificadores e endereços físicos. O comando `hcitool lescan` inicia o varrimento de dispositivos BLE na vizinhança, apresentando os endereços encontrados e, quando disponível, o nome anunciado por cada dispositivo. Identificado o endereço do periférico, estabelece-se a ligação com `gatttool -b <endereço> -I`, que abre a sessão em modo interativo; dentro dessa sessão, o comando `connect` estabelece a ligação efetiva. Segue-se a descoberta dos serviços e características expostos, e a leitura ou escrita dos respetivos valores.

Recomenda-se o seguinte método de trabalho, que evita perder tempo a procurar problemas no sítio errado. Primeiro, preparar e testar o hardware isoladamente, verificando o funcionamento do sketch através do monitor série, antes de envolver a comunicação BLE. Segundo, confirmar a visibilidade do periférico com `hcitool lescan`, garantindo que o módulo está efetivamente a anunciar-se. Terceiro, estabelecer a ligação manualmente com `gatttool` e

explorar os serviços e características disponíveis. Só depois passar ao desenvolvimento em Python, começando pelo varrimento e progredindo para a leitura de características e para as notificações.

A vantagem desta progressão é isolar a origem dos problemas. Uma falha detetada no terceiro passo não pode ser atribuída ao código Python, que ainda não foi escrito; uma falha no segundo passo aponta para a configuração do shield ou para o sketch, e não para a pilha BLE do gateway.

Recapitulando, a programação BLE traduz diretamente a hierarquia GATT, com serviços, características e descritores a terem correspondência nas classes das bibliotecas. Em Linux, a pilha BlueZ é a base sobre a qual assentam as bibliotecas de mais alto nível. O padrão de delegação é o mecanismo através do qual a aplicação reage a eventos assíncronos. E as notificações substituem a interrogação periódica, reduzindo consumo e latência, sendo o mecanismo natural para encaminhar leituras para um middleware de mensagens, tema retomado nas aulas seguintes, quando o papel de cliente passar para um Raspberry Pi a atuar como gateway entre a rede BLE e a rede IP.